

1. Aspectos conceptuales

a) ¿Qué ventajas tiene la utilización de funciones?

REUTILIZAR

→ Escribo la lógica 1 vez y la uso múltiples veces.

MODULARIDAD

→ Simplifica su creación y depuración

LEGIBILIDAD

→ Al estar organizado en bloques con nombres descriptivos es más fácil de leer.

MANTENIMIENTO

→ Si hay un error se corrigen en la definición de la función.

∴ Permiten reutilizar el código para no escribir lo mismo varias veces y ayudan a que sea el código más ordenado y fácil de mantener.

b) ¿Hay algún cuidado en el orden en el que se pasan los parámetros a una función?

Si, el pasaje de parámetros no es conmutativo. Sólo sigue instrucciones posición a posición.

Si yo estoy definiendo una función y escribo en otro orden al atributo va a fallar o no tener sentido porque Python no mueve los datos para que exista la correspondencia adecuada

```
def auto(marca,modelo):  
    print(f"El auto es {marca} {modelo}")  
auto("Ford", "Fiesta")
```

Si llegase a escribir al revés va a decir: El auto es Fiesta Ford.

c) ¿Cuándo uso la sentencia return?

La sentencia return se usa para que una función nos devuelva un resultado y así poder usarlo más adelante.

d) ¿Qué diferencia hay entre la definición y la invocación de una función?

La definición es donde se dan las instrucciones a la función y la invocación es donde se ejecutan las mismas.

e) ¿Qué son los parámetros formales y para qué sirven? Ejemplifique.

f) ¿Qué son los parámetros reales y para qué sirven? Ejemplifique.

Los **parámetros formales** son las "etiquetas" que se otorgan a la función en su definición y los **parámetros reales** son los valores o variables que se le da a esa función cuando se la invoca.

```
def auto(marca,modelo):  
    print(f"El auto es {marca} {modelo}")  
auto("Ford", "Fiesta")
```

Parámetros formales marca y modelo.

Parámetros reales Ford y Fiesta.

g) ¿Qué significa el cuerpo de una función? Ejemplifique.

El cuerpo de una función es el bloque de código que está dentro de la función.

Son las sentencias que se escriben debajo del *def*, las cuales se reconocen al estar indentadas.

```

D  def calcular_precio_final (producto, precio_base, descuento):
E
F      if descuento > 0:
I          rebaja = precio_base * (descuento/100)
N          precio_final = precio_base - rebaja
I
C      else:
I          precio_final = precio_base
O
N      print(f"Procesando el artículo: {producto}")
      return precio_final

```

C
U
E
R
P
O

```

resultado = calcular_precio_final ("Monitor Gamer", 50000, 15)
print (f"El total a pagar es ${resultado}")

```

INVOCACIÓN

Parámetro real

Parámetro formal

h) ¿Existen funciones sin parámetros o argumentos?

Si, existen pero debo respetar la decisión que usé en la definición.

<pre> def saludo(): print("Bienvenidos") saludo() </pre>	<pre> def saludo(palabra): print(f"{palabra} mundo") saludo("Buenas") </pre>
--	--

i) ¿Puede usar una letra o un número como parámetro formal? ¿Y como parámetro real?

No se puede usar **UNA** letra o número como parámetro formal a menos que sea lo que lo defina.

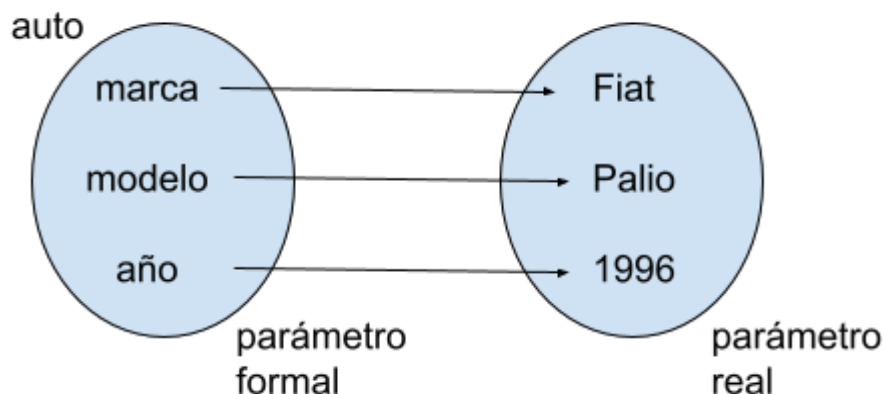
Si puede usarse **UNA** letra o número como parámetro real.

Python no entiende por sí solo que "5" es un número o que "a" es una vocal pero si yo defino la función es_vocal (donde su parámetro formal puede ser letra) o es_impar (donde su parámetro formal es número), y en el cuerpo escribo cuál es la condición de cada uno, cuando escribo la letra o el número específico (parámetros reales) me devuelve si lo son o no.

j) ¿Puedo tener una cantidad distinta de parámetros formales que reales en una función?

No, debe cumplir con la definición de función.

A cada elemento del conjunto de entrada le corresponde uno solo del conjunto de salida.



k) ¿Cómo se puede implementar un módulo con solo definiciones de funciones e importarlo desde tu programa? ¿Cuáles son las formas de importar que ofrece Python?

Puedo tener un módulo con varias funciones y luego en otro módulo puedo importar todo: *import doc* o solo la parte que necesito en ese módulo: *from doc import funcion1*.

Esto me permite que el código sea más limpio y en caso de corrección o mejora sepa donde buscarlo.

l) ¿Qué diferencias hay entre los siguientes códigos?

i) `import math`

ii) `from math import sqrt`

La diferencia es que en `import math` estoy invocando a todas las funciones de la librería mientras que en `from math import sqrt` solo una específica.

Es conveniente usar la primera cuando necesito varias funciones del paquete, evitando escribir líneas de código innecesarias, mientras que la segunda es conveniente cuando necesito pocas.

Esto hace que la ejecución sea más liviana y eficiente, lo cual me hace ahorrar RAM.